BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Võ Thị Như Ý
MSSV / Lớp	3123410439/DTN1241

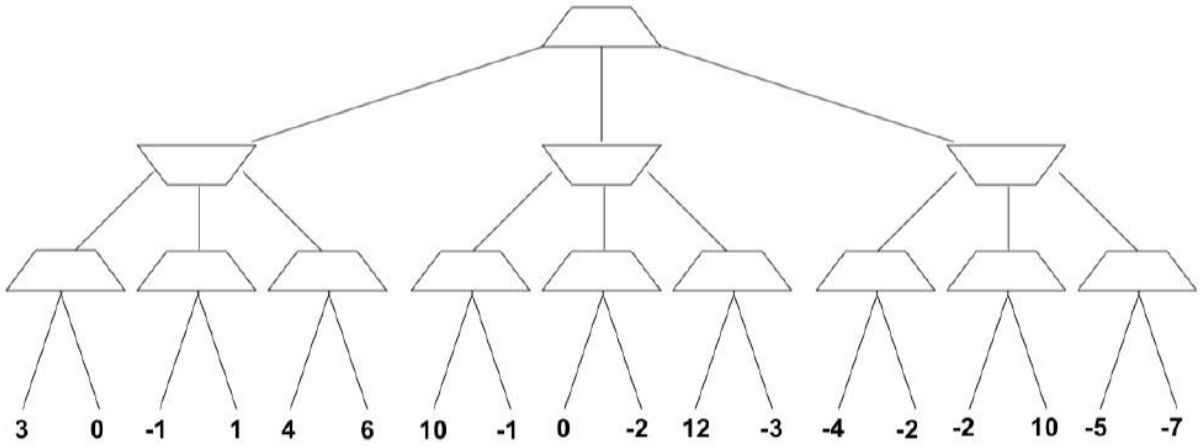
Mục tiêu học tập

- Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa.
- Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.
- Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP.
- Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi.
- Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.



Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

Tầng lá (từ trái sang phải): 3, 0 | -1, 1 | 4, 6 | 10, -1 | 0, -2 | 12, -3 | -4, -2 | -2, 10 | -5, -7

Tầng 3 (MAX - các nút ngay trên lá, lấy giá trị lớn nhất của mỗi cặp lá):

Nút MAX 1: max(3, 0) = 3

Nút MAX 2: max(-1, 1) = 1

Nút MAX 3: max(4, 6) = 6

Nút MAX 4: max(10, -1) = 10

Nút MAX 5: max(0, -2) = 0

Nút MAX 6: max(12, -3) = 12

Nút MAX 7: max(-4, -2) = -2

Nút MAX 8: max(-2, 10) = 10

Nút MAX 9: max(-5, -7) = -5

Tầng 2 (MIN - lấy giá trị nhỏ nhất của các nút MAX con):

Nút MIN bên trái: min(3, 1, 6) = 1

Nút MIN ở giữa: min(10, 0, 12) = 0

Nút MIN bên phải: min(-2, 10, -5) = -5

Tầng 1 (Gốc - MAX, lấy giá trị lớn nhất của các nút MIN):

Nút gốc = max(1, 0, -5) = 1

b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

Nhánh MAX nên chọn: Nhánh bên trái.

Giá trị trò chơi: 1.

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

Nguyên tắc: Duyệt từ trái sang phải với alpha (giá trị MAX đảm bảo tối thiểu) và beta (giá trị MIN giới hạn tối đa). Cắt tỉa khi alpha >= beta.

Quá trình duyệt và cắt tỉa: Nhánh trái: Thăm các lá 3, 0 (thu được MAX = 3), thăm tiếp lá -1, 1 (thu được MAX = 1). Nút MIN bên trái nhận beta = 1. Thăm tiếp lá 4 cho ra giá trị >= 4. Vì 4 >= beta (1), tiến hành cắt tỉa nút lá 6. Nút MIN chốt giá trị = 1, nút gốc cập nhật alpha = 1.

Nhánh giữa: Nút MIN nhận beta = 1 từ gốc. Thăm lá 10 cho ra giá trị >= 10. Vì 10 >= 1, cắt tỉa lá -1 phía sau. Thăm cặp lá 0, -2 cho ra MAX = 0, cập nhật beta của nút MIN thành 0. Tại nhánh MAX tiếp theo, vì beta (0) <= alpha (1), tiến hành cắt tỉa toàn bộ nhánh này (cắt tỉa các lá 12 và -3).

Nhánh phải: Nút MIN nhận beta = 1. Thăm cặp lá -4, -2 cho ra MAX = -2, cập nhật beta thành -2. Vì beta (-2) <= alpha (1), cắt tỉa toàn bộ các nhánh còn lại của nút MIN này (cắt tỉa các lá -2, 10, -5, -7).

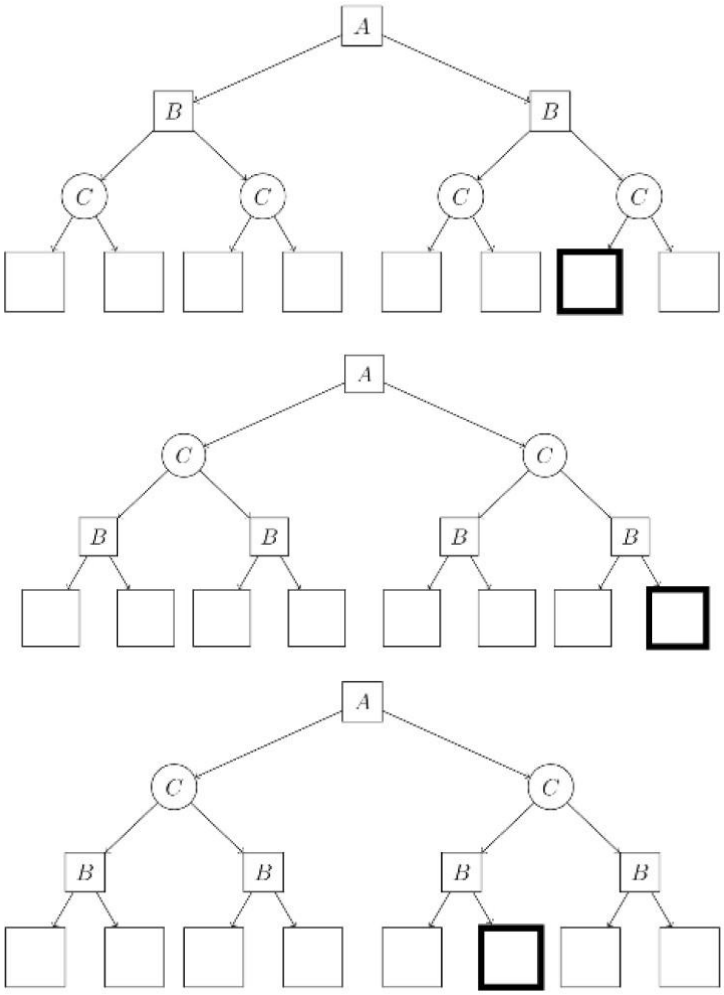
d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.

Các nút lá không cần thăm (bị cắt tỉa): Lá 6, -1, 12, -3, -2, 10, -5, -7 (Tổng cộng có 8 lá bị cắt tỉa).

So sánh: Thuật toán Minimax đánh giá toàn bộ 18 nút lá, trong khi thuật toán Alpha-Beta Pruning chỉ cần đánh giá 10 nút lá.

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.



Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Khả năng cắt tỉa: Không thể.

Giải thích: Nút C tính giá trị trung bình = 0.5 x v1 + 0.5 x v2. Vì các nút lá có thể nhận giá trị rất lớn (tiến tới vô cùng), việc chỉ biết một nhánh con của nút ngẫu nhiên chưa đủ để giới hạn cận trên hay cận dưới của nút đó. Do đó, bắt buộc phải duyệt hết tất cả các lá của nút C viền đậm.

b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Khả năng cắt tỉa: Có thể.

Ví dụ minh họa: Giả sử nhánh C bên trái cho giá trị kỳ vọng là 10. Nhánh C bên phải có nút B đầu tiên đánh giá ra giá trị 5. Khi đó giá trị của C bên phải sẽ phụ thuộc vào nút B đậm còn lại. Nếu nút B đầu tiên bên phải có giá trị lớn (ví dụ bằng 20), giá trị kỳ vọng của C bên phải sẽ chắc chắn >= 10. Nếu nút A (MIN) ở nhánh trái đã tìm được giá trị nhỏ hơn (ví dụ bằng 5), thì C bên phải chắc chắn lớn hơn 5, do đó nút B đậm hoàn toàn có thể bị cắt tỉa.

c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Khả năng cắt tỉa: Có thể.

Giải thích: Tương tự trường hợp 2, nếu giá trị của các nút B đã duyệt ở nhánh phải làm cho giá trị kỳ vọng của nút ngẫu nhiên C vượt quá giá trị mà nút MIN A đã ghi nhận ở nhánh bên trái, nút B còn lại (viền đậm) sẽ bị cắt tỉa.

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

Trong cây Minimax truyền thống, các nút lấy giá trị MIN hoặc MAX tuyệt đối, nên chỉ cần 1 nhánh vượt ngưỡng alpha hoặc beta là có thể khẳng định ngay kết quả. Ngược lại, nút ngẫu nhiên (Chance node) tính giá trị trung bình cộng có trọng số. Một nhánh có giá trị rất thấp vẫn có thể được bù đắp bởi một nhánh khác có giá trị rất cao. Do đó, cơ hội cắt tỉa bị hạn chế hơn rất nhiều và phụ thuộc vào khoảng bù trừ giá trị của các nút lá

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có $V_M \leq V_E$. ☒ Đúng ☐ Sai

Trong Minimax, đối thủ (MIN) luôn chọn hành động tệ nhất cho MAX. Trong Expectimax, đối thủ hành động ngẫu nhiên theo xác suất. Vì kịch bản tồi tệ nhất theo Minimax luôn nằm trong hoặc thấp hơn mức trung bình xác suất của Expectimax, nên V_E luôn lớn hơn hoặc bằng V_M

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M. ☒ Đúng ☐ Sai

Chiến lược Minimax đảm bảo mức bảo vệ tối thiểu là V_M trong mọi trường hợp xấu nhất (worst-case guarantee). Dù môi trường hoặc đối thủ chơi ngẫu nhiên, mức lợi ích kỳ vọng thu được vẫn sẽ luôn lớn hơn hoặc bằng V_M.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E. ☐ Đúng ☒ Sai

Để đạt được lợi ích kỳ vọng cao nhất là V_E, người chơi phải áp dụng chính sách Expectimax tối ưu (tận dụng xác suất của môi trường). Chiến lược Minimax mang tính phòng thủ quá mức nên thường thu về kết quả thấp hơn V_E.

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

☐ Minimax với tiện ích trạng thái cuối +1/−1

☒ Expectimax với tiện ích trạng thái cuối +1/−1

☒ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua

☐ Không phương án nào

d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.

Expectimax rất phù hợp vì bài toán có tính ngẫu nhiên và độ sâu hữu hạn. Value Iteration (thuộc MDP - Quá trình quyết định Markov) giải quyết rất tốt các bài toán ngẫu nhiên có phần thưởng ở trạng thái cuối. Minimax không phù hợp vì không có sự tham gia của đối thủ chủ động chống lại người chơi.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).

Các biến (đỉnh): A, B, C, D, E, F. Các cặp ràng buộc nhị phân (cạnh): (A, B), (A, C), (A, D), (B, C), (B, E), (C, F).

b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Miền biến bị thay đổi: Forward Checking chỉ kiểm tra các biến hàng xóm trực tiếp của A, đó là các biến B, C, D.

c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?

Miền biến bị thay đổi: Kiểm tra các biến hàng xóm chưa được gán của B, đó là các biến C, E.

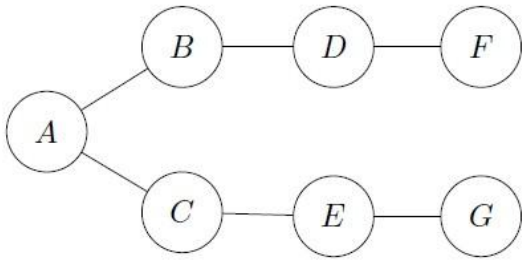
d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?

Miền biến bị thay đổi: Arc Consistency lan truyền ràng buộc trên toàn bộ đồ thị, do đó có thể làm thay đổi miền của tất cả các biến chưa gán còn lại là B, C, D, E, F.

e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?

Miền biến bị thay đổi: Tất cả các biến chưa gán còn lại trong đồ thị là C, D, E, F

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



th ràng bu c cho bài toán th t gán bi n (ngu n: rec3.pdf, trang 6)

e) Thứ tự A → B → C → D → E → F → G

Gán A và B (2 biến chẵn) rồi chạy AC giúp phát hiện mâu thuẫn sớm trên cấu trúc cây. Số lần backtrack trong trường hợp xấu nhất rất ít (khoảng bậc tuyến tính O(d)).

g) Thứ tự G → E → C → A → B → D → F

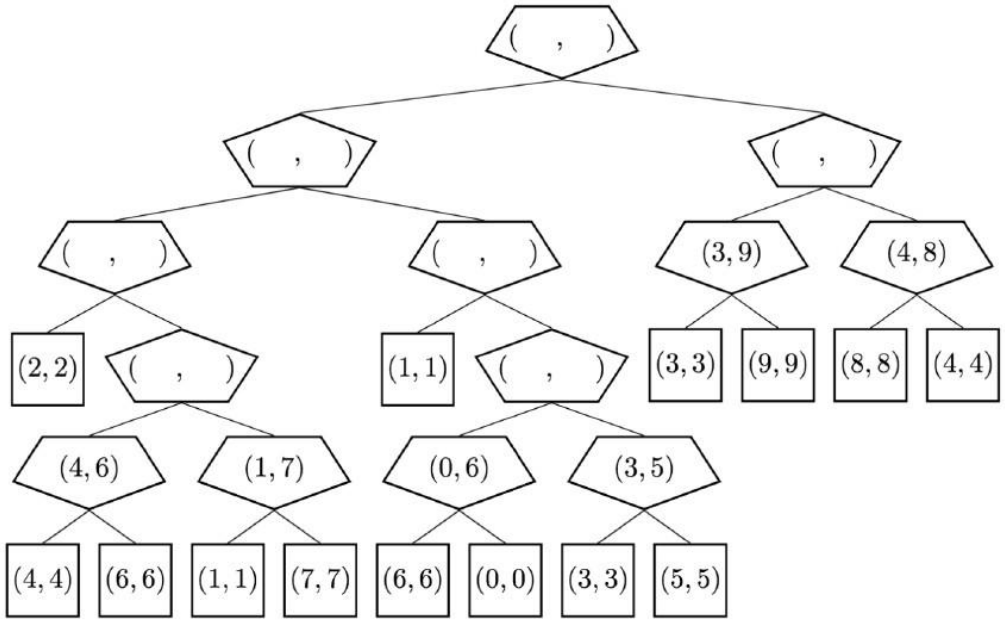
Đi từ lá ngược về gốc. Khi gán 2 biến đầu tiên G và E, AC giúp thu hẹp miền của C. Số lần backtrack xấu nhất ở mức trung bình (khoảng $O(d^2)$ đến $O(d^3)$).

h) Thứ tự G → F → E → D → C → B → A

Thứ tự gán bị nhảy cóc giữa các nhánh ở xa nhau, việc duy trì AC sau 2 biến đầu không hỗ trợ cắt tỉa không gian trạng thái hiệu quả. Số lần backtrack trong trường hợp xấu nhất là lớn nhất (lên tới $O(d^4)$ đến $O(d^6)$).

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.



Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper.

Tầng 4 (Nút Casper nhỏ ở đáy):

Nút dưới trái (lá (4,4), (6,6)): Pacman nghĩ chọn min u = 4, Casper chọn v thực tế lớn hơn là (6,6). Giá trị nút: (4, 6).

Nút tiếp theo (lá (1,1), (7,7)): Giá trị nút: (1, 7).

Nút tiếp theo (lá (6,6), (0,0)): Giá trị nút: (0, 6).

Nút cuối đáy (lá (3,3), (5,5)): Giá trị nút: (3, 5). Tầng 3 (Nút Pacman):

Nút Pacman 1 (con là (4,6) và (1,7)): Pacman chọn max u -> (4, 6).

Nút Pacman 2 (con là (0,6) và (3,5)): Pacman chọn max u -> (3, 5). Tầng 2 (Nút Casper):

Nút Casper trái 1 (con gồm lá (2,2) và nút Pacman (4,6)): Pacman dự đoán min u = 2, Casper thực tế chọn v lớn nhất là (4,6) với v = 6. Giá trị nút: (2, 6).

Nút Casper trái 2 (con gồm lá (1,1) và nút Pacman (3,5)): Giá trị nút: (1, 5).

Nút Casper phải 1 và 2: Đã cho sẵn lần lượt là (3, 9) và (4, 8). Tầng 1 (Nút Pacman):

Nút Pacman trái (con là (2,6) và (1,5)): Pacman chọn max u -> (2, 6).

Nút Pacman phải (con là (3,9) và (4,8)): Pacman chọn max u -> (4, 8). Nút Gốc (Casper):

Con gồm (2,6) và (4,8). Pacman dự đoán giá trị gốc là min u = 2. Casper thực tế chọn nhánh phải vì v = 8 lớn hơn 6. Giá trị nút gốc: (2, 8). Giải thích sự khác biệt giữa u và v: u là giá trị tiện ích mà Pacman lầm tưởng do nghĩ Casper chơi theo kiểu ép điểm nhỏ nhất, còn v là giá trị tiện ích thực tế mà Casper chủ động hỗ trợ để mang lại mức điểm cao nhất cho Pacman

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

Casper nắm rõ cách suy luận của Pacman. Khi Casper nhận thấy một nhánh phía sau sẽ bị Pacman loại bỏ hoàn toàn vì giá trị u không đạt mức tối đa đối với Pacman, Casper sẽ không cần tính toán chi tiết giá trị v của nhánh đó nữa.

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Nếu Pacman biết Casper thân thiện, cả hai sẽ phối hợp hợp tác hoàn toàn (bài toán MAX-MAX). Giá trị toàn bộ cây trò chơi sẽ đạt mức tối đa ở lá có giá trị cao nhất là 9 (tại lá (9,9) ở nhánh phải)

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
Breadth-First Search (BFS)	Đầy đủ nếu yếu tố nhánh b hữu hạn.	Tối ưu nếu chi phí mỗi bước bằng nhau (đơn vị 1).
Depth-First Search (DFS)	Đầy đủ trên đồ thị hoặc cây hữu hạn; Không đầy đủ trên không gian trạng thái vô hạn hoặc có chu trình (trừ khi dùng Graph Search chống lặp).	Không tối ưu trong trường hợp tổng quát.
Iterative Deepening Search (IDS)	Đầy đủ nếu yếu tố nhánh b hữu hạn.	Tối ưu nếu chi phí mỗi bước bằng nhau.
A* Search	Đầy đủ nếu số trạng thái có g(n) <= C* là hữu hạn (chi phí mỗi cạnh lớn hơn epsilon > 0).	Tree Search cần Heuristic h(n) thỏa mãn tính chấp nhận được (Admissible); Graph Search cần Heuristic h(n) thỏa mãn tính nhất quán (Consistent).

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận.

Chi phí bước lớn hơn 0 giúp ngăn chặn các vòng lặp vô hạn có tổng chi phí bằng 0. Tính chất Admissible và Consistent của hàm Heuristic đảm bảo thuật toán A* không bỏ sót đường đi ngắn nhất và không phải mở lại các nút đã đóng.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

a) Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Đường đi tối ưu có thể thay đổi. Cập nhật giá trị g(Y) mới = g(Y) + n, lan truyền chi phí mới này xuống các nút con cháu và tiếp tục mở rộng tập biên (OPEN) nếu cần thiết

b) Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Đường đi tối ưu không đổi. Chỉ cần cập nhật giảm chi phí xuống (C_mới = C_cũ - n), không cần tìm kiếm lại.

c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Đường đi tối ưu không đổi. Giữ nguyên kết quả cũ.

d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Đường đi tối ưu có thể thay đổi vì đường nhánh phụ nay có thể trở nên rẻ hơn đường cũ. Cập nhật lại giá trị g(Y) và đưa Y quay lại tập biên (OPEN) để tiếp tục chạy A*.

e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Đường đi tối ưu không đổi. Giữ nguyên kết quả.

f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Đường đi tối ưu có thể thay đổi. Cập nhật chi phí mới cho cạnh và tiếp tục quá trình tìm kiếm A* từ tập OPEN hiện tại.

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

• A: act(percept) → f_A()

• B: act(percept) → f_B(percept)

• C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

Lớp tác tử phù hợp: B, C.

Giải thích: Môi trường quan sát đầy đủ và tĩnh giúp trạng thái hiện tại cung cấp đủ mọi thông tin cần thiết, tác tử phản xạ đơn giản (B) hoặc có bộ nhớ (C) đều hành động rất tốt.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

Lớp tác tử phù hợp: C.

Giải thích: Môi trường quan sát một phần đòi hỏi tác tử phải lưu giữ lịch sử nhận thức quá khứ để suy luận ra trạng thái ẩn của thế giới.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

Lớp tác tử phù hợp: C.

Giải thích: Tương tự câu b, tác tử cần duy trì lịch sử nhận thức để cập nhật trạng thái niềm tin (belief state) và đưa ra quyết định tối ưu dưới tính ngẫu nhiên.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.

Đáp án: Đúng.

Ví dụ minh họa: Môi trường không có lựa chọn (No-op environment) - nơi môi trường chỉ có duy nhất một hành động hợp lệ duy nhất (hoặc mọi hành động đều đem lại kết quả và mức thưởng hoàn toàn giống hệt nhau). Bất kể tác tử thuộc lớp nào (A, B hay C) chọn hành động gì đi nữa thì đó vẫn luôn là hành động hợp lý tối ưu trong môi trường đó.

— Hết —